Design Phases of the bnn_fcc Accelerator

Author Name
Course or Lab
Gainesville, Florida, USA
email@example.com

1 Summary

1.1 Results

Table 1 summarizes the explored design points. The first three columns identify the architectural choice for each run: input bus width, parallel input consumption, and the per-layer neuron split. The remaining columns are the judging metrics: FMAX, throughput, latency, FPS, and resource utilization. The blank FPS and resource fields are placeholders for the final submission measurements once the selected point is locked.

Table 1: Current design-space results summary.

Input Bus Width	Parallel Inputs	Parallel Neurons	FMAX	Throughput	Latency	FPS	Resource Utilization
64	32	—	722	105	290.2	—	—
128	64	64,32,10	635	56.1	195.2	—	—
256	128	64,64,10	501	32.2	130.2	—	—
512	128	256,64,10	—	17.8	95.2	—	—
1024	128	256,64,10	—	10.6	78.4	—	—

1.2 Implementation and Tradeoffs

The implementation target was to maximize throughput first and then optimize for clock frequency inside that higher-throughput design space. In practice, the design work started by widening the data path and increasing layer-local parallelism so more work could be issued per cycle. After that, the RTL was reworked to recover timing with banked RAMs, replay buffering in the compute path, and pipelined fanout registers in the configuration path. The main tradeoff is therefore straightforward: push the architecture toward more useful parallel work first, then add the structure needed to keep FMAX competitive.

1.3 Verification/Testbench

The baseline directed testbench still provides the initial pass/fail check, and the verification flow was then extended with UVM to cover behavior beyond a single nominal configuration and inference sequence. The extended environment was used to verify the following additional functionality:

- Implemented a UVM flow to extend the original testbench coverage.
- Supports partial `TKEEP` configuration and input streams.
- Supports reconfiguring weights and thresholds after a few images.
- Supports interleaved configuration streams, including thresholds-first and out-of-order layer updates.
- Supports deeper hidden-layer stacks; see `make sim-ten-hidden`.
- Tested long delay gaps and extreme weight, threshold, and pixel values.
- Scales to higher neuron counts when the input bus width increases.

2 Top-Level Organization

This report is organized in the same order that the design is used in practice. The top level accepts a configuration stream that loads weights and thresholds, an input stream that carries activation data, and an output path that returns the classification result. The main body then follows two phases of operation: the configuration phase and the compute phase.

At the top level, `rtl/bnn_fcc.sv` connects three major subsystems: `config_manager`, `data_in_manager`, and `bnn`. The first prepares and distributes model parameters, the second reshapes incoming input data into the binary words expected by the network, and the third contains the replicated layer datapaths that perform inference.

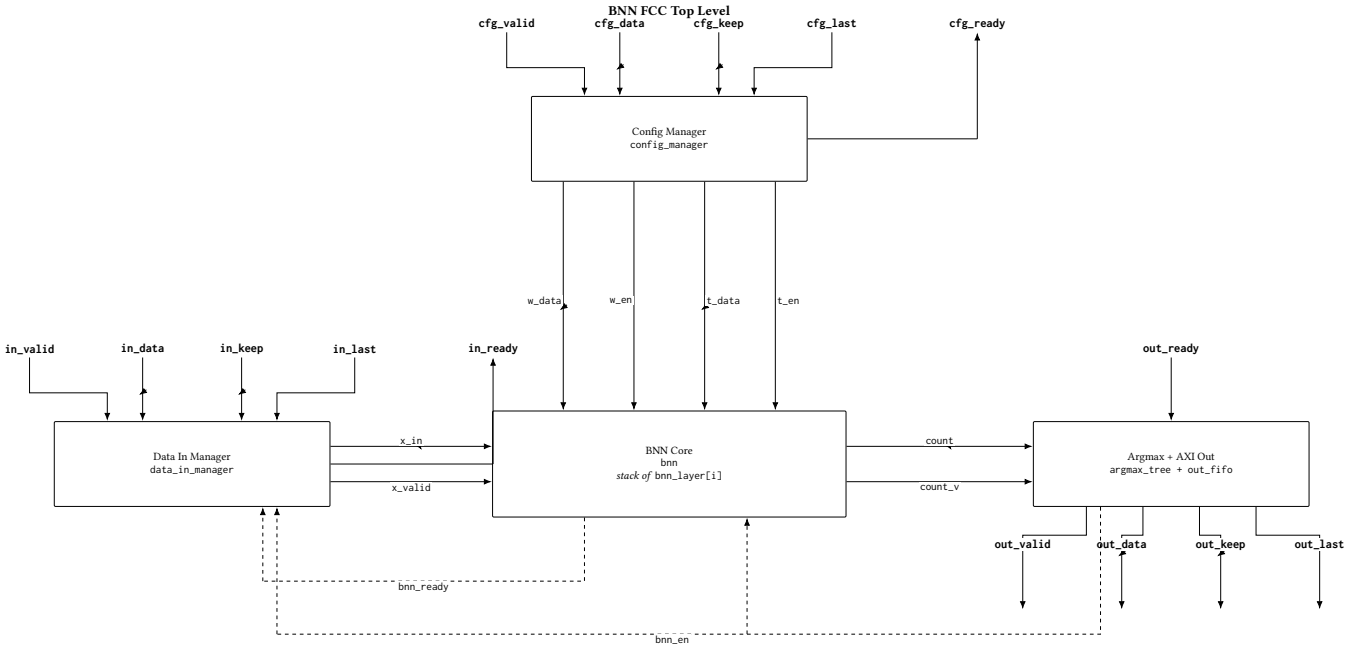

Figure 1: Top-level organization of `rtl/bnn_fcc.sv`. Configuration ingress, activation ingress, the BNN core, and the output side are shown with the main interface signals.

The top-level structure is intentionally streaming-oriented. Rather than converting everything into a single global control machine, each phase reshapes the incoming stream into a form that is locally useful to the next stage. That same idea appears again inside the configuration and compute phases, where staging FIFOs and fanout pipeline registers are used to keep the control problem manageable and the timing paths short.

3 Configuration Phase

The configuration phase runs before inference and is responsible for converting the serialized parameter stream into aligned per-layer writes. In this design, that work is intentionally split into two levels. First, the top-level `config_manager` reshapes the incoming byte stream and separates weight payloads from threshold payloads. Then each `bnn_layer` uses a local `config_controller` path to distribute those writes across replicated RAM banks.

3.1 Configuration Manager

The top-level configuration datapath is implemented in `rtl/config_manager.sv` with support from `tkeep_byte_compactor`, `vw_buffer`, `config_manager_parser` and `config_manager_pad_fsm`. Its job is to accept the external configuration stream, compact away invalid bytes, recover message structure, and emit write-oriented streams that are already separated into the weight and threshold paths.

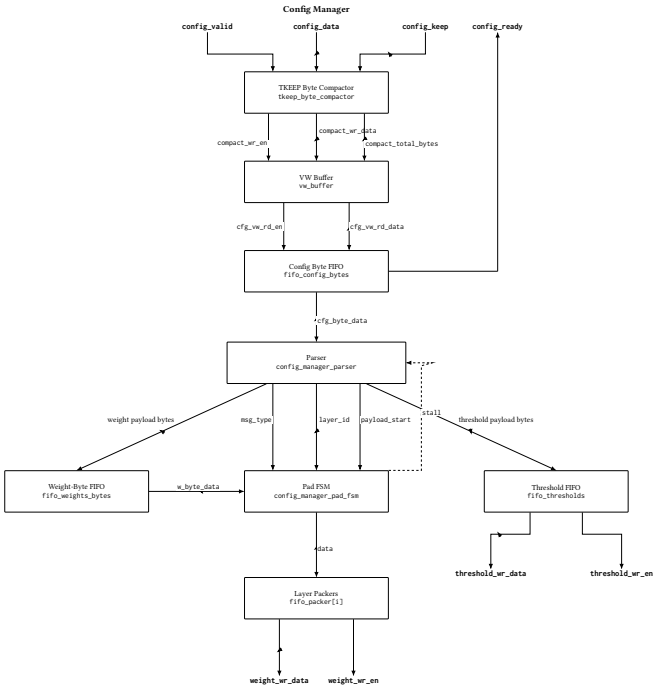

Figure 2: Block diagram of the `config_manager` datapath and its primary interface signals.

The front end of this path is byte-oriented because the external interface can arrive with sparse valid-byte patterns. The parser then restores the higher level meaning of the stream by decoding the message header, choosing the weight or threshold destination, and generating the control metadata needed by the padding logic. On the weight side, the pad state machine inserts synthetic bytes so the downstream layer packers can emit aligned write words.

3.2 Layer-Local Write Distribution

After the top-level parser finishes that global reshaping step, each `bnn_layer` still has to fan the write stream out to replicated local memories. That is the purpose of `config_controller` and the surrounding staging registers in `rtl/bnn_layer.sv`. The controller generates the bank-select and address signals, while the aligned data stream bypasses the controller datapath and is retimed for fanout.

Those fanout pipeline registers are a central part of the implementation, not a minor detail. They cut the long controller-to-BRAM distribution path into a manageable stage while preserving a steady stream of writes after the pipeline fills. That is why this report treats the local write-distribution block as its own step in the overall configuration phase.

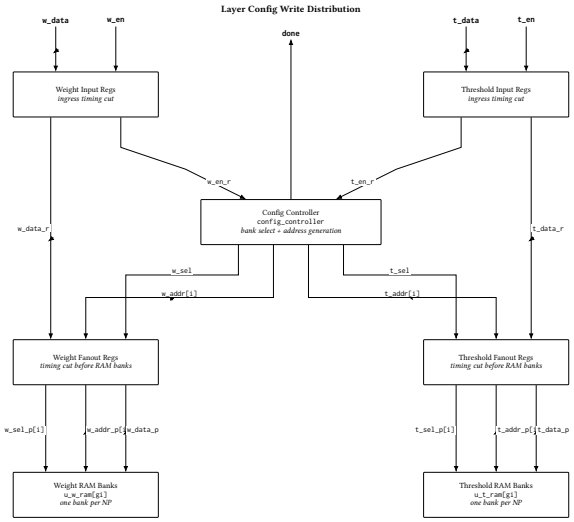

Figure 3: Layer-local config-write distribution around `config_controller`. The highlighted fanout pipeline registers are the timing-cut stage that drives the per-bank weight and threshold RAMs.

4 Compute Phase

Once the model parameters are loaded, the design switches to the compute phase. This phase again has two main pieces. The first is the input-preparation path in `data_in_manager`, which converts the incoming activation stream into the binary words expected by the network. The second is the layer-local forward datapath inside `bnn_layer`, where buffered activations, RAM reads, and neuron-processor outputs are coordinated.

4.1 Input Preparation

`rtl/data_in_manager.sv` mirrors several of the same structural ideas used in the configuration path. It begins with byte compaction on the external stream, repacks the resulting bytes into bus-width words, applies the local frame and padding rules needed by the design, binarizes pixel values, and then stages the resulting binary words before handing them to the BNN core.

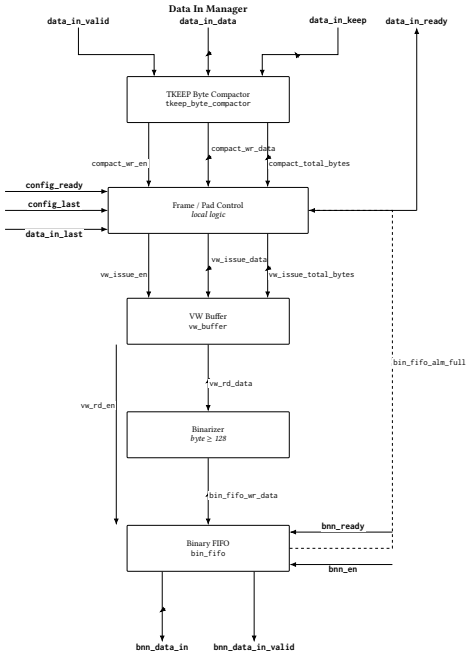

Figure 4: Block diagram of the `data_in_manager` datapath and its primary interface signals.

This preprocessing stage is important because it decouples the external input stream shape from the internal datapath width. By the time activations leave `data_in_manager`, they already match the first layer's expected binary format and can be replayed locally without repeatedly revisiting the original byte-level protocol.

4.2 Layer Compute Flow

Inside `rtl/bnn_layer.sv`, the forward path is organized around the replay buffer, the read-side control logic, the banked parameter memories, and the neuron-processor array. The intent is to keep the activation stream and the shared RAM-read schedule locally synchronized while still exposing the timing registers that matter to the implementation.

The replay buffer and align-delay logic keep the activation side available when the local controller issues parameter reads. In parallel, `neuron_controller` sequences the weight and threshold-bank access pattern, and the neuron processors combine those operands to generate the next layer's outputs. Together, these blocks form the runtime half of the design that follows the earlier configuration phase.

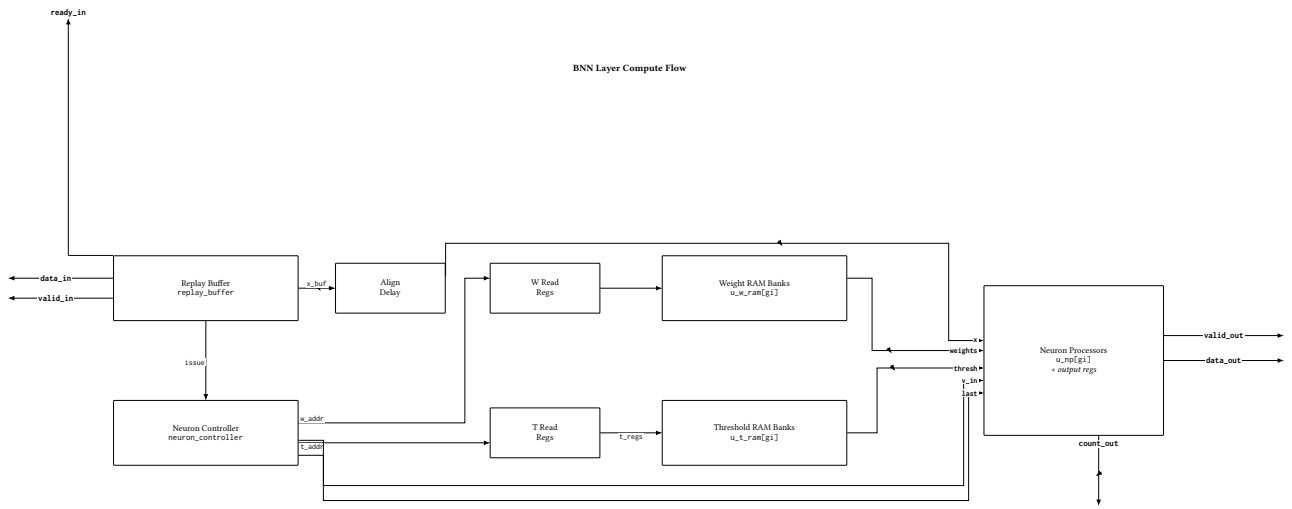

Figure 5: Simplified compute flow inside one `bnn_layer`. The replay buffer, read-control path, RAM banks, and neuron-processor array are shown in the order they participate in inference.

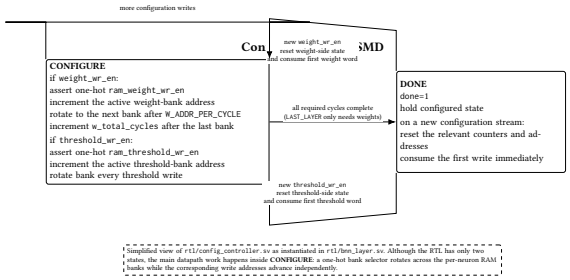

Figure 8: Simplified FSMD view of the layer-local `config_controller`.

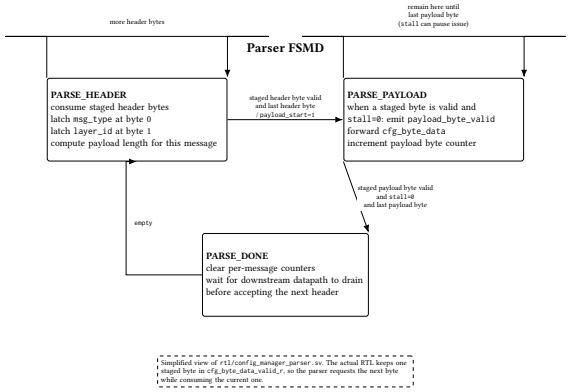

Figure 6: Simplified FSMD view of `config_manager_parser`.

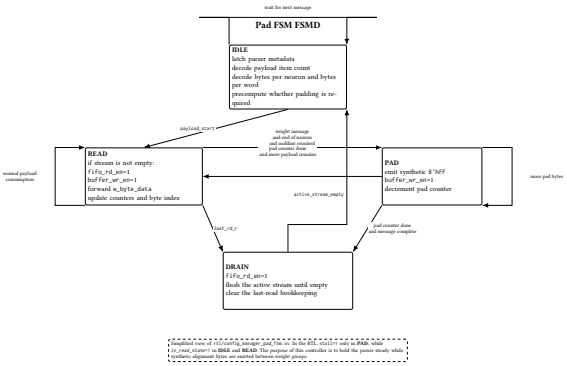

Figure 7: Simplified FSMD view of `config_manager_pad_fsm`.

A Supplementary FSMD Views

The following control-state diagrams are intentionally simplified. They do not attempt to reproduce every temporary register or default assignment in the RTL. Instead, they summarize the dominant control flow behind the configuration path so the higher-level block diagrams in the main text remain focused on the data movement.